Trabalho Prático Nº 2 — Sistema de Vigilância e Alerta Ambiental

Universidade do Minho · LEI · Comunicações por Computador · 2026/2027
8 aulas · grupos de 3 alunos

1. Contexto

A monitorização ambiental em larga escala — deteção de incêndios, qualidade do ar, meteorologia — assenta em redes de sensores dispersos por território com conectividade irregular, e em múltiplos consumidores da mesma informação: bombeiros, proteção civil, autarquias, equipas de investigação. Cada consumidor quer um subconjunto diferente dos dados, e a mesma leitura pode ter de chegar a vários destinos ao mesmo tempo.

Este padrão — produtores que não conhecem os consumidores, consumidores que declaram interesses, um intermediário que faz corresponder uns aos outros — designa-se **publicação/subscrição** (*publish/subscribe*). É a base do MQTT, do AMQP ou do Kafka, e distingue-se do modelo cliente-servidor por não existir uma relação 1:1 entre quem envia e quem recebe.

Neste trabalho cada grupo desenvolve, de raiz, um sistema de publicação/subscrição completo e os dois protocolos aplicacionais que o suportam.

1.1 Cenário

Uma rede de **sensores** publica leituras periódicas (temperatura, humidade, partículas) e eventos pontuais (ignição, falha de sensor). Um conjunto de **centros de operação** subscreve as categorias que lhe interessam. Um **broker** faz a mediação.

Nem toda a informação tem o mesmo valor. Uma leitura de temperatura registada duas vezes não altera nada: é o mesmo valor, repetido. Um alerta de ignição entregue em duplicado pode desencadear dois despachos de meios para o mesmo local. O sistema tem de tratar os dois casos de forma diferente, e é esse o cerne do trabalho.

1.2 Como se faz e como se avalia, em resumo

O trabalho faz-se em **quatro fases**, ao longo das oito aulas (secção 6): um sistema que arranca e entrega uma publicação; os protocolos e a fiabilidade numa rede que perde pacotes; um sistema de publicação/subscrição com múltiplos subscritores e entrega *exactly-once*; vários brokers, ligados aos vizinhos pelos mesmos dois protocolos. Entre a aula 4 e a aula 5 há uma **avaliação intercalar**.

A avaliação final tem cinco componentes: o **contrato de execução**, cumprido ou não (secção 5.2), a **solução** — o código, nas funcionalidades e na qualidade —, o **relatório** (secção 8.2), a **demonstração** e as **perguntas** (secção 7).

2. Objetivos

- Desenhar e implementar **dois protocolos aplicacionais** sobre transportes distintos, com separação clara entre **plano de controlo** e **plano de dados**.
- Implementar, ao nível aplicacional, **mecanismos de fiabilidade** sobre um transporte não fiável: numeração de sequência, confirmações, retransmissão, temporizadores e deteção de duplicados.
- Implementar **garantias de entrega** distintas sobre a mesma infraestrutura.
- Lidar com ***fan-out*** — entrega a múltiplos destinatários —, mantendo estado de entrega independente por subscritor.
- Avaliar o sistema em **condições adversas**: perda de pacotes, e um subscritor que deixa de responder sem fechar a sessão.

- **Estender os dois protocolos aos brokers vizinhos**, para entregar através de mais do que um broker, e comparar o custo do *flooding* com o de propagar subscrições.
- Utilizar *sockets* TCP e UDP diretamente.

3. Arquitetura

O sistema tem três tipos de entidade e dois protocolos (Figura 1). Nas três primeiras fases há um único broker; na quarta há três, ligados em linha (secção 3.5).

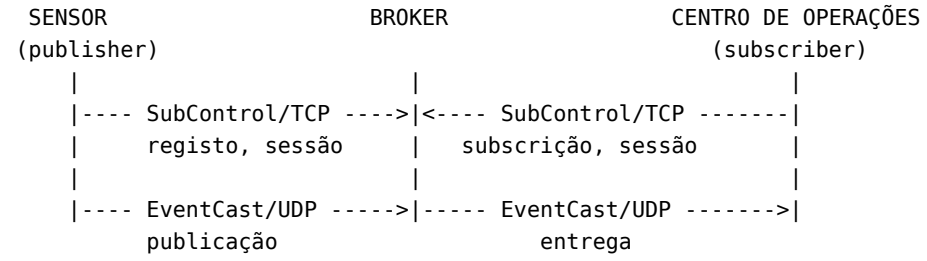


Figura 1. As três entidades e os dois protocolos: o SubControl sobre TCP, o EventCast sobre UDP.

Sem interface. O broker, o publicador e o subscritor são processos sem interface de utilizador — sem menus, TUI nem GUI, sem nada lido do teclado. Arrancam pelo `run.sh`, fazem o que a configuração lhes diz — o publicador gera as publicações que a chave `publish` pede, com conteúdo sintético — e terminam sozinhos. Tudo o que se vê do sistema está nos registos e na rede. **Entregar** uma publicação é escrever a sua linha no registo do subscritor (secção 5.2).

3.1 SubControl — plano de controlo, sobre TCP

O SubControl trata tudo o que é pouco frequente e não pode falhar em silêncio: estabelecimento de sessão, identificação de clientes, registo de publicadores, subscrição e cancelamento de subscrição, terminação ordenada.

O TCP fornece uma *stream* de bytes, **não** uma sequência de mensagens. Cabe ao grupo definir e implementar a delimitação de mensagens sobre essa *stream*.

3.2 EventCast — plano de dados, sobre UDP

O EventCast transporta as publicações do sensor para o broker e as entregas do broker para cada subscritor interessado. O UDP não garante entrega, ordem nem ausência de duplicados: **todos os mecanismos de fiabilidade são implementados ao nível aplicacional pelo grupo**.

3.3 Mensagens

Nos dois protocolos, **toda a informação de controlo — tipo de mensagem, identificadores, números de sequência, indicadores de estado, comprimento — viaja num cabeçalho binário desenhado pelo grupo, construído e interpretado byte a byte.** O conteúdo transportado — a leitura, o texto do alerta — é opaco para o protocolo e pode ser codificado como o grupo entender. É assim que estão construídos o IP, o TCP, o RTP e o próprio MQTT, e é sobre o cabeçalho que incidem as perguntas: quantos bytes por campo, o que acontece quando um número de sequência dá a volta, o que o recetor faz com um cabeçalho truncado.

Por isso, não é permitido usar bibliotecas de serialização (`pickle`, Protocol Buffers, MessagePack, JSON e equivalentes) para produzir ou interpretar cabeçalhos; bibliotecas ou implementações de publicação/subscrição (MQTT, AMQP, ZeroMQ, Kafka e equivalentes); e bibliotecas que forneçam fiabilidade sobre UDP. A comunicação é feita diretamente sobre *sockets* TCP e UDP. A **leitura** da especificação do MQTT é encorajada: é o exemplo acabado de tudo o que aqui se desenha — cabeçalho fixo binário, comprimento de tamanho variável, trocas de subscrição e publicação, um

four-way handshake para *exactly-once* — e mostra a forma sem dar a resposta, porque o MQTT vai buscar a fiabilidade ao TCP e o vosso plano de dados não pode.

3.4 Tópicos

O espaço de nomes e as regras de correspondência **são fixados pelo enunciado**, não escolhidos pelo grupo: os tópicos e os filtros chegam ao sistema pela configuração (secção 5.2).

Um **tópico** de publicação tem sempre três níveis, `regiao/estacao/metrica`, sem caracteres universais. Um **filtro** de subscrição é um tópico completo ou um prefixo terminado em `/*`:

```
north/station7/temperature   uma métrica de uma estação
north/station7/*             todas as métricas dessa estação
north/*                      tudo o que vem dessa região
```

O carácter universal aparece apenas no fim — `north/*/temperature` não é um filtro. Com profundidade fixa, não se pergunta se `north/*` corresponde a `north`: nada é publicado em `north`.

Filtros sobrepostos entregam uma cópia, não uma por filtro. Um subscritor com `north/station7/temperature` e `north/station7/*` recebe cada publicação uma vez. Esta regra diz respeito à correspondência, não à retransmissão, e vale nos dois níveis de garantia: em G1 uma publicação *retransmitida* é entregue as vezes que chegar (secção 4), mas uma publicação recebida uma só vez e correspondida por três filtros é entregue uma só vez. Uma publicação que corresponda a filtros sobrepostos com `qos` diferentes é entregue uma vez, com o mais forte deles, `qos=2` — entregá-la uma só vez cumpre o filtro G1, e o contrário não —, e nunca acima do `qos` do fluxo (secção 4).

Onde a cópia única é decidida é decisão vossa, e tem consequências. Decidida no broker, segue um datagrama; decidida no recetor, segue um por filtro correspondido e descarta os excedentes depois de terem atravessado a rede. As duas cumprem a regra; só uma poupa a rede. A diferença vê-se numa captura (secção 8.2).

Continua a ser decisão do grupo, e a justificar no relatório, como o tópico viaja no cabeçalho e que estrutura de dados faz a correspondência.

3.5 Vários brokers — o que os vizinhos trocam

Na quarta fase correm três brokers, cada um na sua LAN (secção 5.1), ligados em linha (Figura 2): `broker1 – broker2 – broker3`. Cada publicador e cada subscritor liga-se ao broker que a sua configuração indica, na chave `broker_control`. Uma publicação feita em `broker1` para um subscritor de `broker3` atravessa dois saltos de broker, e `broker2` tem de a reencaminhar em nome de um subscritor que nunca viu.

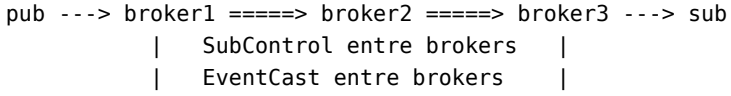


Figura 2. Três brokers em linha: `broker2` reencaminha em nome de um subscritor que nunca viu.

O que é dado. Cada broker recebe na configuração (secção 5.2) uma chave `peer` por vizinho, com o plano de controlo dele — e só dos vizinhos imediatos. Não recebe a malha, nem um mapa, nem a lista dos subscritores dos outros.

A rede de brokers é lógica. Dois brokers são vizinhos porque a configuração os liga por `peer`, não por estarem perto na rede: a linha `broker1 – broker2 – broker3` é uma rede sobreposta (*overlay*) à da secção 5.1, e o caminho IP entre dois vizinhos é o que os *routers* escolherem. `broker1` e `broker3` não são vizinhos, embora a rede os ponha a duas ligações um do outro.

O que está fixado. Entre brokers correm os mesmos dois protocolos, não um terceiro (Figura 2). Dois vizinhos ligam-se pelo SubControl: o que inicia a sessão fá-lo como qualquer outro cliente e anuncia-se como broker, e cada um fica a saber nessa sessão a porta de dados do outro (secção 5.2). O controlo — a sessão, e o que os brokers digam uns aos outros sobre subscrições — segue pelo SubControl; as publicações seguem pelo EventCast, confirmadas entre vizinhos como entre o

broker e um subscritor. As regras de cabeçalho da secção 3.3 valem para tudo o que acrescentarem aos dois.

O que tem de acontecer.

- Cada broker estabelece sessão com os vizinhos que a configuração lhe dá, e só com esses.
- Uma publicação chega a todos os subscritores interessados, em qualquer broker, com a garantia pedida.
- Uma subscrição feita **depois** de o sistema estar a correr, num broker distante, passa a receber a partir daí — o que um broker sabe sobre os outros não é construído uma só vez no arranque.

A forma mais simples é o *flooding*, e está correta. Um broker que reencaminhe cada publicação para todos os vizinhos exceto aquele de onde ela veio — *split horizon* — entrega tudo a todos. Sem essa exceção, `broker1` e `broker2` devolvem um ao outro cada publicação, indefinidamente. O *flooding* tem um custo: uma publicação que ninguém subscreve atravessa a linha inteira. A alternativa é propagar as subscrições — quando um subscritor de `broker3` subscreve `north/*`, `broker2` e `broker1` ficam a saber que publicações com esse prefixo seguem na direção de `broker3` — e reencaminhar só para onde há interesse. Isso desloca custo do plano de dados para o plano de controlo: cada subscrição passa a viajar entre brokers. **As duas coisas medem-se numa captura, ligação a ligação.** Um sistema que não faz *flooding* fez um trabalho melhor, e nas perguntas espera-se que saibam dizer quando é que deixa de compensar.

O que é decisão vossa. As mensagens que acrescentam ao SubControl e ao EventCast, e o seu formato; *flooding*, propagação de subscrições, ou uma combinação dos dois; se um filtro é anunciado tal como chegou ou agregado — anunciar `north/*` em vez de dois `north/stationN/*` é *route summarization*, com as vantagens e os custos que tem na camada de rede; como a tabela de encaminhamento é guardada e consultada; o que acontece a uma subscrição cancelada; com que garantia uma publicação viaja entre brokers; o que fazer a um vizinho que deixa de confirmar; qual dos dois vizinhos inicia a sessão, se ambos se têm como `peer`. Um filtro de subscrição é um prefixo, e uma tabela de encaminhamento sobre prefixos é uma tabela de rotas: aprendida com um vizinho, consultada por cada publicação que chega, com um próximo salto que é um endereço.

Dois identificadores. Uma publicação que atravessa três brokers é transmitida por três emissores diferentes. O identificador que diz *"esta confirmação responde a que transmissão"* só faz sentido entre dois vizinhos; o que diz *"esta é a publicação tal"* viaja inalterado do publicador ao subscritor. São dois campos com dois âmbitos.

4. Garantias de entrega

Cada fluxo de publicação e cada subscrição declaram a garantia pretendida. O sistema suporta **dois níveis** (Tabela 1):

Tabela 1. As duas garantias de entrega.

Nível	Designação	Semântica exigida
G1	*at-least-once*	Toda a mensagem publicada é entregue a todos os subscritores do tópico. O recetor **não mantém estado de deteção de duplicados**: uma publicação que chegue duas vezes fica registada duas vezes.
G2	*exactly-once*	Toda a mensagem publicada é entregue a todos os subscritores do tópico, e **o registo do subscritor nunca tem a mesma publicação duas vezes**.

G1 não é um mínimo a superar. Um sistema que elimine duplicados em G1 tem um só mecanismo com dois nomes: entrega *exactly-once* em ambos os níveis, e a diferença entre G1 e G2 — o eixo deste trabalho — deixa de existir. Numa rede que perde pacotes, um subscritor G1 acaba por registar alguma publicação duas vezes; se nunca o faz, está a eliminar duplicados. Para quem escreve o recetor, a decisão de consultar ou não a tabela de duplicados depende da garantia da entrega — é uma linha de código, e é nela que está a diferença.

A formulação de G2 é deliberada. Numa rede que perde pacotes não é possível garantir que uma mensagem seja *transmitida* exatamente uma vez. O que se exige é que o subscritor nunca a *registe* repetida — identificação de mensagens e eliminação de duplicados no recetor, sobre um mecanismo de retransmissão que, esse sim, transmite as vezes que forem precisas. A distinção é matéria de avaliação.

Na configuração as garantias chamam-se `qos`: `qos=1` é G1, `qos=2` é G2. O publicador declara a garantia de cada fluxo, na chave `publish`, e o subscritor a de cada subscrição, na chave `subscribe` (secção 5.2). **Uma entrega tem a mais fraca das duas**: quem publica decide quanto vale o que publica, e uma subscrição não a pode elevar. O recetor tem de saber a garantia de cada entrega — como o sabe é desenho vosso. **Simplificação aceite:** um sistema pode assumir que o fluxo e todas as subscrições que lhe correspondem têm o mesmo `qos` — e então a garantia de cada entrega é simplesmente essa.

A ordem não é garantida. Nem G1 nem G2 dizem nada sobre ordem. Numa rede que perde pacotes a ordem perde-se de facto sempre que há mais do que uma publicação em trânsito: uma retransmissão chega depois das que lhe seguiam. Não tentem preservá-la, a menos que seja decisão vossa e a saibam justificar.

Os mecanismos concretos — formato das mensagens, algoritmo de retransmissão, temporizadores, estado por subscritor, *sliding window* ou *pipelining*, agregação — são **inteiramente decisão do grupo**.

5. Topologia e contrato de execução

5.1 Topologia

Quatro *routers* ligados em anel e cinco LAN (Figura 3): os publicadores, os subscritores e cada um dos três brokers têm a sua. Nenhum broker partilha a LAN com clientes nem com outro broker, pelo que todo o tráfego entre um cliente e um broker, e entre dois brokers, atravessa pelo menos uma ligação entre *routers*.

Esta secção é uma especificação, não uma descrição. Cada grupo constrói esta topologia no CORE da sua máquina virtual — nós, endereços, ligações — e é nela que testa o seu sistema.

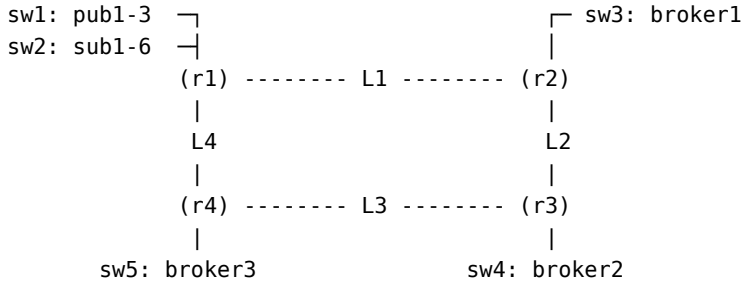


Figura 3. A topologia: quatro *routers* em anel e cinco LAN, uma por papel.

Os publicadores e os subscritores estão ambos em `r1`, em LAN diferentes. Nas três primeiras fases corre um único broker, `broker1`, em `r2`: uma publicação vai do publicador ao broker por `L1` e volta ao subscritor pela mesma ligação, no sentido contrário. Na quarta fase correm os três brokers: uma publicação de `broker1` para um subscritor de `broker3` faz `L1`, `L2`, `L3` e `L4` — cada salto entre brokers é exatamente uma ligação, e a volta ao anel fecha-se no subscritor.

Nós, endereços e ligações. Os *hosts* são nós `PC` do CORE e os *routers* nós `router`, com os serviços que o CORE lhes dá por omissão. Cada LAN é um *switch* do CORE. Os endereços estão na Tabela 2 e as ligações na Tabela 3: máscara `/24` em toda a parte e, em cada ligação, a extremidade *a* em `.1` e a *b* em `.2`.

Tabela 2. Os endereços dos nós nas cinco LAN: `sw1` é `10.0.1.0/24`, `sw2` `10.0.2.0/24`, `sw3` `10.0.3.0/24`, `sw4` `10.0.4.0/24` e `sw5` `10.0.5.0/24`.

nó	endereço	nó	endereço
r1	10.0.1.1 e 10.0.2.1	r3	10.0.4.1
pub1–pub3	10.0.1.10–12	broker2	10.0.4.20
sub1–sub6	10.0.2.31–36	r4	10.0.5.1
r2	10.0.3.1	broker3	10.0.5.20
broker1	10.0.3.20		

Tabela 3. As quatro ligações entre *routers*.

ligação	a	b	rede
`L1`	r1	r2	10.0.11.0/24
`L2`	r2	r3	10.0.12.0/24
`L3`	r3	r4	10.0.13.0/24
`L4`	r4	r1	10.0.14.0/24

Encaminhamento. É o OSPF dos *routers* do CORE que o faz: não há tabelas a escrever. Basta que todas as LAN se alcancem umas às outras — confirmem-no com `ping` e `traceroute` entre *hosts* de LAN diferentes antes de testarem o vosso sistema, porque o OSPF leva alguns segundos a convergir depois de a sessão do CORE arrancar. Nada no vosso sistema depende do caminho que um pacote segue: os pares opostos do anel — `r1` e `r3`, `r2` e `r4` — têm dois caminhos do mesmo comprimento, e qualquer um serve.

A topologia é fixa — a mesma para todos os grupos, em todos os turnos — e o vosso sistema não a conhece nem depende dela: recebe endereços e portas pelo ficheiro de configuração (secção 5.2) e mais nada. Os endereços estão aqui para construírem a rede, não para os escreverem no código. A liberdade de desenho deste trabalho está no protocolo, não na cablagem.

5.2 Contrato de execução

O sistema é lançado, configurado e observado através de três formatos — a linha de comandos, o ficheiro de configuração e o registo de entregas — e só desses. São a fronteira do sistema, igual para todos os grupos; o que está dentro dela é desenho vosso.

O contrato é obrigatório, e é uma componente da classificação: pequena e binária. Não há meio-termo, porque não há meio contrato: sem ele, o sistema não pode ser executado nem lido por quem não o escreveu.

O cumprimento verifica-se nas duas entregas, a da avaliação intercalar e a final: o código entregue é executado por um **banco de testes automático**, em ensaio este ano, através deste contrato e sobre a topologia da secção 5.1. Cumprir o contrato significa que os três papéis arrancam pelo `run.sh`, invocado a partir de outra diretoria, com a configuração que lhes é dada; terminam como diz o parágrafo **Sinais**, abaixo; e os registos que escrevem têm o formato e o relógio desta secção. Tudo o resto que o banco de testes observa não entra na classificação: serve à equipa docente para preparar as interações seguintes com o grupo.

Cada grupo entrega um executável ou *script* com a seguinte interface fixa:

```
./run.sh broker     --id <id> --config <ficheiro>
./run.sh publisher  --id <id> --config <ficheiro>
./run.sh subscriber --id <id> --config <ficheiro>
```

O `run.sh` resolve os seus próprios caminhos. Não pode assumir a diretoria de trabalho com que é invocado:

```
here=$(cd "$(dirname "$0")" && pwd)
exec python3 "$here/src/main.py" "$@"
```

Os caminhos que recebe de fora — o `--config` e a chave `log` — são sempre absolutos. Tem permissão de execução e terminações de linha LF: com CRLF, a primeira linha não é um interpretador válido e o `run.sh` não arranca.

Linguagem de implementação: C, C++, Python, Java ou Go. O `run.sh` corre numa imagem que fornece apenas estes compiladores e interpretadores, sem gestores de pacotes nem acesso à rede: tudo aquilo de que o sistema precisa está no arquivo entregue.

Compilação. Se a linguagem precisar de ser compilada, é o `run.sh` que compila: a equipa docente não compila nada, e corre o arquivo tal como foi entregue. Os três papéis são invocados ao mesmo tempo, a partir da mesma cópia do arquivo, pelo que o `run.sh` compila uma vez só — ainda que três invocações o peçam em simultâneo — e não volta a compilar se nada mudou.

Sinais. Cada processo termina por si quando se esgota o tempo da chave `duration` — ou, no publicador, da chave `drain`; um processo que não termine recebe `SIGTERM` e, 5 s depois, `SIGKILL`. Um subscritor que deixa de responder sem fechar a sessão é simulado com `SIGSTOP` ao seu grupo de processos e retomado com `SIGCONT` ao mesmo grupo. Para isso, **tudo o que o `run.sh` lança fica no grupo de processos com que foi invocado** — nada de `setsid`, *daemons* ou processos desligados.

Ficheiro de configuração

Escrito por quem lança o sistema, lido pelo vosso sistema no arranque. **Não é JSON**: o formato é deliberadamente trivial de interpretar em qualquer linguagem, incluindo C. Uma linha é `chave = valor`, e `#` inicia um comentário até ao fim da linha. Um valor é um primeiro termo obrigatório seguido de opções `nome=valor`. As chaves estão em inglês, como o código e os artefactos do trabalho.

Todos os valores vêm de fora: o sistema não tem valores por omissão. Cada chave da Tabela 4 aparece exatamente uma vez na configuração dos papéis que a usam; cada chave da Tabela 5 aparece zero ou mais vezes, e as repetições formam uma lista pela ordem em que aparecem. As opções de cada valor estão sempre todas presentes, e a configuração não traz chaves além destas. Todas têm de estar implementadas: um teste que não precise de uma chave dá-lhe na mesma um valor. A Figura 4 mostra uma configuração de cada papel.

Tabela 4. As chaves que aparecem exatamente uma vez na configuração de cada papel que as usa.

chave	papéis	significado
`bind`	broker	endereço onde escutar
`control_port`	broker	porta TCP do plano de controlo
`data_port`	broker	porta UDP do plano de dados
`broker_control`	publicador, subscritor	`endereço:porta` do plano de controlo do broker a que se liga
`log`	publicador, subscritor	caminho **absoluto** do ficheiro onde escrever o registo — de publicações ou de entregas; a diretoria já existe
`duration`	broker, subscritor	segundos até terminar
`start_after`	publicador	segundos a esperar, **após estabelecer a sessão**, antes de começar a publicar
`drain`	publicador	segundos, **no máximo**, a continuar a retransmitir após a última publicação; termina antes se tudo foi confirmado
`evict_after`	broker	segundos que um subscritor pode passar sem confirmar nada, havendo algo por lhe entregar, até o broker o expulsar

Tabela 5. As chaves que aparecem zero ou mais vezes.

chave	papéis	significado
`peer`	broker	`endereço:porta` do plano de controlo de um broker vizinho — quarta fase
`publish`	publicador	um fluxo: `<topico> qos=<1|2> count=<n> rate=<por segundo>`
`subscribe`	subscritor	`<filtro> qos=<1|2>`, aplicado no arranque
`at`	subscritor	uma ação aos `<segundos>`, contados **a partir do arranque do processo**, de um de dois tipos: `<segundos> subscribe=<filtro> qos=<1|2>` — subscreve, como a chave `subscribe`; **OU** `<segundos> unsubscribe=<filtro>` — cancela a subscrição desse filtro

```
# broker1 — em 10.0.3.20
bind         = 10.0.3.20
control_port = 7310
data_port    = 7311
duration     = 60
evict_after  = 5
# pub1 — em 10.0.1.10
broker_control = 10.0.3.20:7310
log            = /tmp/tp2/pub1.log
start_after    = 2
drain          = 10
publish        = north/station7/temperature qos=1 count=200 rate=10
publish        = north/station7/humidity qos=2 count=100 rate=5
# sub1 — em 10.0.2.31
broker_control = 10.0.3.20:7310
log            = /tmp/tp2/sub1.log
duration       = 45
subscribe      = north/station7/humidity qos=2                  # no arranque
at             = 20 unsubscribe=north/station7/humidity         # aos 20 s, cancela
at             = 25 subscribe=north/station7/temperature qos=1  # aos 25 s, subscreve
```

Figura 4. Um exemplo de configuração de cada papel, com um só broker; na quarta fase, cada broker recebe também uma chave `peer` por vizinho.

As chaves que mais facilmente se tomam por decorativas:

- **As portas do broker não são fixas.** Um sistema que fixe 5000 no código funciona com uma configuração e falha com a seguinte. Os clientes recebem apenas o plano de controlo do broker: a porta de dados chega-lhes pelo SubControl, ao estabelecer a sessão — e o mesmo entre brokers vizinhos. Os endereços são IPv4, os da Tabela 2; não há resolução de nomes.
- **`start_after`** existe para que todos os processos estabeleçam a sessão em paralelo e só a publicação espere. Sem ele, uma publicação pode partir antes de um subscritor ter concluído a subscrição.
- **`drain` é a `duration` do publicador.** Um publicador que termine assim que a última chamada a `publish()` retorne perde as retransmissões em curso — numa rede sem perdas não se nota.
- **`start_after` e `at` contam a partir de momentos diferentes** — a sessão e o arranque do processo, respetivamente. Para que uma subscrição temporizada aconteça depois de a publicação ter começado, o valor de `at` tem em conta a `start_after` dos publicadores.
- **`evict_after`** é o único prazo que o broker recebe de fora. Um subscritor que deixe de responder durante menos do que `evict_after` mantém a sessão e recebe tudo; um que deixe de responder durante mais é expulso — o broker fecha-lhe a sessão TCP e descarta o que tinha em fila para ele — passado esse prazo, nem antes nem muito depois. Um broker que expulse antes perde um subscritor vivo; um que nunca expulse acumula filas para subscritores que já não existem, e continua a retransmitir para eles. Um subscritor expulso não tem de voltar, mas não entrega nada publicado depois de a sessão ser fechada.

Registos

Cada subscritor escreve, no ficheiro indicado em `log`, **uma linha por publicação entregue**:

```
<timestamp_ms> <id> <topico> <message_id>
```

`timestamp_ms` é o **tempo Unix em milissegundos** — o relógio do sistema, não o tempo desde o arranque nem um contador monótono. `id` é o `--id` com que o processo foi lançado. Cada linha chega ao ficheiro no momento em que é escrita — *flush* a cada linha —, para que nenhuma forma de terminar a perca, `SIGKILL` incluído.

Cada publicador escreve, no mesmo formato e no seu próprio ficheiro, uma linha por mensagem publicada. **É esse registo que serve de referência**: o que os subscritores entregaram confere-se com o que os publicadores dizem ter publicado. O `message_id` identifica a publicação de ponta a ponta e tem de ser a mesma cadeia de caracteres nos dois registos. Dois publicadores nunca partilham um tópico, pelo que o par `<topico> <message_id>` identifica uma publicação nos dois registos.

O broker não escreve registo: o que ele faz vê-se na captura.

6. Fases e calendário

O trabalho faz-se em quatro fases, cada uma sobre a anterior (Tabela 6). Cada fase diz o que o sistema faz no fim dela e o que acrescenta ao relatório (secção 8.2): o relatório escreve-se melhor ao longo do trabalho do que no fim.

Tabela 6. As quatro fases: o que o sistema faz no fim de cada uma, e o que cada uma acrescenta ao relatório.

Fase	No fim, o sistema…	No relatório
1. Contrato e primeira publicação	arranca pelo `run.sh` a partir de qualquer diretoria e lê a configuração; os clientes recebem a porta de dados do broker pelo SubControl (secção 5.2), e uma publicação vai do publicador ao broker e chega ao subscritor; os dois registos estão no formato e no relógio da secção 5.2	a arquitetura
2. Protocolos e fiabilidade sob perda	estabelece a sessão SubControl antes de usar o EventCast — o broker responde antes do primeiro datagrama de cada cliente; com um publicador, um broker e um subscritor, entrega tudo em G1 (secção 4) numa rede que perde pacotes, com o mecanismo de fiabilidade que o grupo escolheu — *stop-and-wait*, *sliding window* com *go-back-N* ou com *selective repeat*; expulsa um subscritor que deixou de confirmar ao fim de `evict_after`	os campos e as trocas do SubControl e do EventCast; o mecanismo e os temporizadores, com os seus valores, e os testes que os põem à prova
3. Vários subscritores e G2	em G2, o registo de cada subscritor nunca tem a mesma publicação duas vezes, e cada entrega tem a garantia que lhe cabe (secção 4); faz *fan-out* para vários subscritores, sem que um que deixou de confirmar atrase os restantes; entrega a cada um só o que os seus filtros pedem, e uma só cópia com filtros sobrepostos (secção 3.4)	a eliminação de duplicados, e os testes que mostram o G2, o *fan-out*, a seletividade e a independência das entregas
4. Vários brokers	aceita subscrições e cancelamentos durante a execução (`at`); três brokers ligados por `peer` entregam a um subscritor a dois brokers de distância, incluindo a uma subscrição feita depois do arranque (secção 3.5)	o que os brokers trocam, e o custo de cada ligação entre brokers

Cada grupo faz o seu próprio plano, com uma referência: um grupo que chega à avaliação intercalar com a segunda fase fechada está a tempo. Um que não chega deve rever o plano e falar disso com o docente, na aula ou por email.

Há duas entregas (Tabela 7). A data da avaliação intercalar é marcada pelo docente de cada turno, entre a aula 4 e a aula 5.

Tabela 7. As entregas e os seus prazos.

Entrega	O quê	Prazo
Avaliação intercalar	o código	23h59 do dia da avaliação intercalar
Final	o código e o relatório	23h59 do dia da aula 8

Os prazos contam-se a partir das aulas do vosso turno.

7. Avaliação

7.1 Como se obtém a classificação

A avaliação final tem cinco componentes: o **contrato de execução** (secção 5.2), a **solução**, o **relatório** (secção 8.2), a **demonstração** e as **perguntas**.

A solução é o código entregue, avaliado nas funcionalidades — o que o sistema faz, fase a fase (secção 6) — e na qualidade: a estrutura, a clareza e a robustez do código.

A demonstração é da responsabilidade do grupo, feita na vossa própria máquina, perante o docente. O grupo escolhe o que mostra e como o mostra. Uma captura aberta ao lado da especificação do vosso cabeçalho é a forma mais direta de mostrar um protocolo a funcionar.

As perguntas incidem sobre o vosso próprio código e sobre os temas de todas as fases, independentemente da fase a que o grupo chegou. A classificação é explicada a quem a solicitar.

7.2 Assistentes de geração de código

A utilização de assistentes baseados em modelos de linguagem é **permitida e esperada**. Não é pedido que declarem o que usaram, nem isso seria verificável.

O que se avalia não é a autoria do código mas a compreensão do sistema. A demonstração mostra o que o vosso sistema faz; as perguntas, o que cada um percebeu do porquê. Um sistema que funciona e cujos autores não conseguem explicar o comportamento dos seus próprios temporizadores numa rede que perde pacotes não obtém a classificação que o seu funcionamento sugeriria.

8. Entregáveis

8.1 Código

O trabalho é desenvolvido em **repositório Git**. Cada entrega (secção 6) é feita na plataforma de e-learning, como arquivo `CC-TP2-PL<Turno>-G<Grupo>.zip` com a seguinte estrutura mínima:

```
/src             código
/run.sh          contrato de execução, compilação incluída (secção 5.2)
/relatorio.pdf   o relatório (na entrega final)
```

O `run.sh` está na raiz do arquivo, e tudo aquilo de que o sistema precisa para compilar e correr está lá dentro.

8.2 Relatório

Acompanha o código na entrega final, no **formato LNCS** — *Lecture Notes in Computer Science*; instruções para autores em https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines — com cerca de **14 páginas**, um limite indicativo. Contém:

- **a arquitetura**: os processos, o que cada um mantém em estado e como se ligam;

- **as decisões de desenho**, cada uma com a sua justificação;
- **a especificação dos protocolos** — o SubControl e o EventCast, incluindo o que lhes acrescentaram entre brokers:
  - os campos de cada mensagem: nome, tamanho em bytes e significado;
  - os mecanismos: delimitação de mensagens, sessão, confirmação e retransmissão, eliminação de duplicados, expulsão, o que os brokers trocam;
  - os temporizadores: quais são, com que valores, e o que acontece quando expiram;
  - diagramas de sequência de uma execução simulada — um *dry run* — com o valor dos campos em cada mensagem;
- **os testes executados e os seus resultados**: para cada um, as condições da rede, a carga e o que se observou nos registos e na captura.

Todas as figuras e tabelas têm legenda e são referidas no texto.

Os registos só provam o que o teste lhes pede. Uma publicação que chega de ponta a ponta mostra que o caminho existe, não que chegam todas quando são muitas e seguidas: é com várias em trânsito ao mesmo tempo que aparecem os problemas de um broker que trata uma mensagem mas não um *burst*. Do mesmo modo, uma rede que não perde pacotes não põe à prova nenhum mecanismo de fiabilidade.

Uma captura mostra o que os registos não provam, e é aí que estão os testes mais informativos:

- a sessão TCP com o broker antes do primeiro datagrama de cada cliente;
- o mecanismo de fiabilidade visto de fora: quantos datagramas seguem antes de o outro lado responder, e o que é retransmitido depois de uma perda — só o que se perdeu, ou tudo o que vinha a seguir;
- nenhum datagrama para um subscritor que não pediu o tópico, ou que já o cancelou;
- os datagramas recebidos por entrega num subscritor com filtros sobrepostos (secção 3.4);
- na quarta fase, os datagramas por publicação e os pacotes de controlo em cada ligação entre brokers, comparados com o mínimo necessário para chegar aos subscritores interessados.